1 of 723

Request for Team 101 Capstone Review 2 Approval

Inbox ×

NAVYA G N PESU-EC-B.Tech 2023 6:11 PM (2 minutes ago)

Good evening ma'am, Requesting your approval for the capstone review 2 slides attached below.

Dr. M. Farida Begam PESU-EC-CSE 6:13 PM (0 minutes ago)

to me

Approved for Team 101

Thanks, and regards,
Dr. M. Farida Begam,
Professor, Department of CSE
IEEE Senior Member, ISTE, & ISCA
PES University EC Campus,
Electronic City,
Bengaluru - 560100.
Ph: +91-8220876618

•••

UE23CS320A- Project Phase - 1

Review -2

Project Title : AI-Assisted Test Case Generation for Mobile and Web UI/UX Applications

Project ID : 101

Project Guide : Dr.Farida Begam

Project Team : Navya G N PES2UG23CS372
Nidhi K PES2UG23CS383
Nikita Kolathaya PES2UG23CS387
Sandeep PES2UG23CS525

Agenda

1. Introduction and Motivation
2. Problem Statement
3. Abstract and Scope
4. Suggestions from Review – 1
5. Research / Technology gap and Challenges
6. Objectives
7. Literature Survey / Existing System
8. References

Introduction and Motivation

Modern software applications — especially web and mobile platforms — are becoming increasingly **dynamic and interactive**. Testing these applications manually is **time-consuming, repetitive, and error-prone**. Even with automated testing tools, writing and maintaining test scripts still requires **significant human effort**.

Our project aims to address this problem by introducing an **AI-assisted test case generation system** that uses **large language models (LLMs)** to automatically generate functional test cases from real web or mobile interfaces.

Introduction and Motivation

- Manual test design slows down software release cycles.
- Automated test script creation requires technical expertise.
- With the rise of AI and multimodal models (which understand images and text), there is an opportunity to **make testing more intelligent, faster, and accessible**.
- Our motivation is to **reduce human effort** in test creation and to **increase testing efficiency and coverage**.

Problem Statement

Traditional software testing relies on manual/rule-based test case generation, struggling with modern web/mobile interfaces. No lightweight, AI-assisted tool exists to:

- Automatically explore web/mobile app interfaces.
- Understand their structure (screenshots, DOM).
- Generate functional executable test cases

This project uses LLM reasoning to automate test case generation, eliminating predefined scripts, datasets, or deep reinforcement learning.

Abstract and Scope

Main Objective : To design and develop an AI-assisted testing system that:

- Connects to a live web or mobile application,
- Explores and understands the interface, and
- Generates and executes a sequence of actions.
- Automatically generates functional test cases for each page visited.

In Scope

- Functional test case generation
- Browser automation using Playwright
- LLM integration for analyzing web pages and generating tests.
- Simple dashboard interface for input and results display.

Abstract and Scope

Out of Scope

- UX or accessibility analysis.
- Performance or security testing.
- Automated test execution in external frameworks.
- Custom model training or fine-tuning of LLMs.
- Native mobile app testing.

Abstract and Scope

We adopted a **modular and scalable architecture** using Python (or Node.js) as the backend, integrating a **Large Language Model (LLM)** for reasoning.

System Components:

1. **Browser Controller:** Automates browser actions (open page, click, take screenshot, get DOM).
2. **LLM Service:** Sends screenshots(optional) + DOM to a text LLM (like GPT-4 or 3.5).
   The LLM interprets the json and suggests next actions or generates test cases.
3. **Main Orchestrator:** Coordinates between the browser and the LLM. Manages exploration flow and stores visited pages.
4. **Dashboard Interface:**Lets users input a URL and view the generated test cases

Suggestions from Review -1

- **Remark:** The idea was not clearly defined and seemed too ambitious.

  Action Taken: We refined the project scope to focus only on *functional test case generation* using LLMs and browser automation, removing the UX and accessibility parts.
- **Remark:** The dataset or input source was not clearly explained.

  Action Taken: We defined that the system will use *open-source or demo web applications* and collect their *DOM and screenshots* as input data.
- **Remark:** Review relevant research papers to strengthen the methodology.

  Action Taken: We studied recent research on *AI-assisted and RL-based web testing*, which helped refine our approach and make it more realistic.

Challenges and Research Gap

Challenges

- Dynamic web pages make automated exploration and understanding difficult.
- LLMs may hallucinate.
- Generating accurate and relevant test cases from screenshots and DOM requires precise LLM interpretation.
- Lack of standardized datasets for evaluating AI-based testing systems.

Challenges and Research Gap

Research Gap

1. Underexplored Application of LLMs in UI Test Case Generation: Current research predominantly utilizes code-based or model-based testing. The direct application of Large Language Models (LLMs) to analyze web interfaces (via screenshots and DOM) for the generation of functional test cases remains largely unexplored.
2. Absence of LLM-driven Browser Exploration Frameworks: Existing automation approaches primarily depend on rule-based crawlers or reinforcement learning agents. There is a clear need for integrating LLM reasoning with browser automation tools such as Playwright or Selenium to enable intelligent exploration and comprehension of web applications.
3. Limited Generation of Human-Readable (BDD-style) Test Cases: Conventional automation tools typically produce low-level scripts. Research into combining AI with Behavior-Driven Development (BDD) for the automated creation of clear, readable test cases (e.g., in Gherkin format) is still quite scarce.

Architecture / System Module

Visual Flow URL → Controller → Capture → LLM Service → Orchestrator → Dashboard.

Architecture: Practical Agentic Hybrid (Playwright + LLM Service) Core concept – Combine automation + reasoning + reporting.

1. **1. Browser Controller:**
   - Automates browser using Playwright/Selenium.
   - Functions: openPage(), takeScreenshot(), getDOM(), executeAction().
2. **2. LLM Service:**
   - Communicates with GPT-4 or 3.5 via API.
   - Functions: getExplorerAction(), getTestCases(), getUx Suggestions().
3. **3. Main Orchestrator:**
   - Central “brain” coordinating modules.
   - Implements exploration loop + stores memory.
4. **4. Dashboard:**
   - HTML/JS frontend for user input and output.
   - Displays screenshots, test cases, UX reports.

Objectives

1. **Build a Node.js/Python backend connecting Playwright + LLM:** implement an orchestrator that controls Playwright, sends screenshots/DOM to the LLM, receives JSON actions and test outputs, and logs all interactions for replay.
2. **Implement an autonomous exploration loop:** run a repeatable loop that opens pages, captures DOM/screenshots, asks the LLM “what to do next”, executes the returned action, and stores the visited-page memory.
3. **Generate functional test cases :** prompt the LLM to produce test cases for each saved page.
4. **Develop a dashboard to display results and screenshots:** provide a simple UI to start runs, view exploration paths, inspect screenshots/DOM and see test results.
5. **Benchmark LLM accuracy and exploration depth, and evaluate outputs:** run experiments across models and measure Execution Success Rate, Bug Detection Precision, Exploration Depth, and Coverage to compare reliability and trade-offs

Literature Survey/Existing System

Literature Review

- A critical assessment of the research conducted on the topic.
- Include 5-6 recently published research papers/products.
- Summarize each paper/product with appropriate detail based on its relative importance to the overall literature.

Title :	End-to-end RPA-like testing using reinforcement learning By C Păduraru · 2024
Methodology	The paper uses three deep reinforcement learning approaches: They create application graphs (manually or via crawling) that abstract UI states and flows, then train agents to navigate these graphs while testing functionality
Findings/Research	Agents can generate tests 6x faster than humans (430 tests/hour vs 72 tests/hour) and discovered 2 real bugs plus 8 artificially injected ones that human testers missed.
Future work	Future work includes better integration with industrial RPA workflows and expanding beyond the single banking application they tested.
Datasets used	UiPath's UiBank application (a mock banking app)
Gaps	Tested only one simple application. Relies heavily on manual application graph creation and developer-provided hints. No visual UI understanding - only uses DOM structure, missing layout and UX issues. Artificially injected bugs rather than discovering real usability problems

Title :	"End-to-End Automated UI Testing Workflow for Web Sites with Intensive User-System Interactions" By Ramazan Faruk Oguz and his colleagues
Methodology	A 5-step workflow to automatically generate test scripts for new websites: **1. Crawling:** A bot maps all pages and links. **2. Topic Tagging (Regex):** URLs are tagged with simple "topics" (e.g., `Home`, `Product`, `Cart`). **3. "Imagination Engine" (SeqGAN):** A GAN (Generator vs. Discriminator) creates new, realistic topic sequences. **4. Finding the Path (HMM + Viterbi):** An HMM (Hidden Markov Model) calculates URL probabilities, and the Viterbi algorithm finds the most likely URL path for the GAN's new topic sequence.**5. Running the Test:** The new, generated URL path is sent to a tool (like Selenium) to run the test.
Findings/Research	The AI created realistic new user paths. **Key Finding:** 81% of generated paths were **new** and **more statistically probable** than ones real users took. It successfully finds **new, high-risk test paths** that humans haven't.
Future work	Use "word embeddings" (better AI) for tagging, not just Regex. More model optimization.
Datasets used	Real data from 3 sites: an e-commerce store, Loadium, and Testinium
Gaps	**Problem:** How to test a new site at "time zero" (t_0) with no user data. **Limitations:** The method depends on clean URLs; topic tagging is very simple.

Title :	"LLM-powered Multimodal Insight Summarization for UX Testing"Kelsey Turbeville, Ranjitha Kumar, Jennarong Muengtaweepongsa, et al. (2024)
Methodology	**Two-stage LLM prompting:** (1) generate multimodal transcripts per session (2) aggregate insights across sessions. Combines verbal, behavioral, and design data (clickstream, DOM, audio). Chrome extension used for data capture; GPT-3.5 and GPT-4 models leveraged. Insights traceable to source evidence.
Findings/Research	Processed 75,464 UX studies; 3,821 researchers created 56,830 insight summaries. Improved efficiency and analysis speed with linked evidence verification. **Challenges include LLM arithmetic limits and transcription errors**. Customization needs identified based on user feedback.
Future work	Improve counting accuracy; allow hiding/pinning insights; add user-customizable prompts; explore multimodal LLMs for direct video/audio processing; enable insight editing; improve export features; address transcription quality; customize for varied research domains.
Datasets used	Real UserTesting platform data: 75,464 UX studies over 8 months capturing multimodal streams (clickstream, surveys, screen recordings, audio transcription), DOM tree data, event logs, and researcher feedback.
Gaps	Limited LLM counting/arithmetic precision; transcription inaccuracies; lack of customization; no editing or advanced export; dependency on text representation may miss some visuals; limited evaluation beyond feedback; potential hallucinations; no comparison with manual methods; scalability concerns.

Title :	An Autonomous RL Agent Methodology For Dynamic Web UI Testing in A BDD Framework by Ali Hassan Mughal- Mar 2025
Methodology	- Models the website as a maze (pages = nodes, interactions = transitions). - Uses Reinforcement Learning (RL) - Defines start points (homepage/login) and endpoints (confirmation/error). - Auto-generates BDD test cases (Gherkin) from successful runs. - Includes state representation (DOM, vision encoders, text), action space, reward mechanisms, and integration with CI/CD.
Findings/Research	- RL agents achieve **higher defect detection** and **broader test coverage** compared to scripted tests. - Reduction in **manual effort** due to automatic BDD test generation. - Demonstrated feasibility on open-source web applications.
Future work	- Extend to **more complex web environments**. - Improve **state representation** and **reward mechanisms**. - Explore **transfer learning** and **multi-task learning** for better generalization. - Deeper integration with **multimodal LLMs** for richer perception of UIs.
Datasets used	Evaluated on **open-source web applications**. - Literature references mention **WebArena**, **Mind2Web**, **World of Bits**, and **VisualWebArena** as realistic environments for training/evaluation.
Gaps	No **fully unified framework** yet combining RL, multimodal perception (vision + text), and BDD. - Scalability and **real-world deployment** still limited. - Current RL-based systems lag behind **human-level performance** in UI testing. - Need for more **robust datasets** and evaluation benchmarks.

Title :	Vision-driven Automated Mobile GUI Testing via Multimodal Large Language Model (VisionDroid) -July 2024 , **Publication:** *Proceedings of the 47th International Conference on Software Engineering (ICSE 2025)* — ACM/IEEE
Methodology	VisionDroid integrates screenshots and view-hierarchy data to enable **vision-driven, non-crash bug detection** in Android apps. It uses GPT-4 as a multimodal reasoning engine to analyze GUI images and textual metadata, simulate user actions, and detect logic or display inconsistencies automatically. The system autonomously explores app UIs, learns functional relationships between screens, and generates structured bug reports without human intervention.
Findings/Research	- Achieves **significant gains in code and activity coverage** (+21–33%) over baseline tools. - Detects **non-crash logic and UI bugs** that rule-based or vision-only testers miss. - Demonstrates **real-world bug discovery** (29 new bugs found, 19 fixed). - Validates that combining **image + view-hierarchy + LLM reasoning** improves precision (up to 76%) and recall (up to 64%) in automated GUI testing.
Core Techniques	- **Multimodal Large Language Models (LLMs):** GPT-4 used for vision + text reasoning. - **Vision Prompt Construction:** Bounding-box overlays align widgets with textual properties. - **Function-Aware Exploration:** Uses LLM reasoning to plan actions and avoid repetition. - **Logic-Aware Bug Detection:** Clusters GUI sequences to identify inter-page logical inconsistencies. - **In-Context Learning:** Retrieves similar bug reports using Word2Vec for contextual guidance.
Applications	Automated **mobile app GUI testing** for detecting logic and usability bugs. Suitable for **non-crash functional testing**, visual consistency validation, and **AI-assisted QA workflows**.
Gaps	- High token and computation cost for multimodal inputs. - Limited to Android UI testing; generalization to other platforms remains unverified. - Relies on accurate screenshot–DOM alignment; errors here propagate to reasoning.

Title :	Intent-Driven Mobile GUI Testing with Autonomous Large Language Model Agents (2024)
Methodology	Proposed a **multi-agent cognitive architecture (Planner, Actor, Observer, Reflector)** using GPT-3.5 and GPT-4 to perform **intent-driven GUI testing** on Android apps. The system perceives the GUI hierarchy (XML tree), plans user-like “intents” (goals), executes actions through ADB, observes results, and reflects to improve future decisions. Introduces **memory modules (short-, long-, and spatial-term)** and an **intent hierarchy** for goal-oriented testing instead of random exploration.
Findings/Research	Demonstrated that LLM-based agents can autonomously generate **realistic user tasks** and execute them on mobile apps without prior training. Achieved higher **activity and feature coverage** than baseline random or script-based explorers.
Future work	It could be extended to handle **multi-app workflows**, i.e., tasks that **span across multiple apps**.
Datasets used	Tested on a **set of real Android applications** (20–30 apps) from open-source repositories like **F-Droid**
Gaps	Relies heavily on **textual GUI metadata**, so it cannot detect **visual or UX-related issues** (contrast, alignment, readability). Lacks **true reinforcement learning or self-improvement**; reflection memory is heuristic, not continuously optimized.

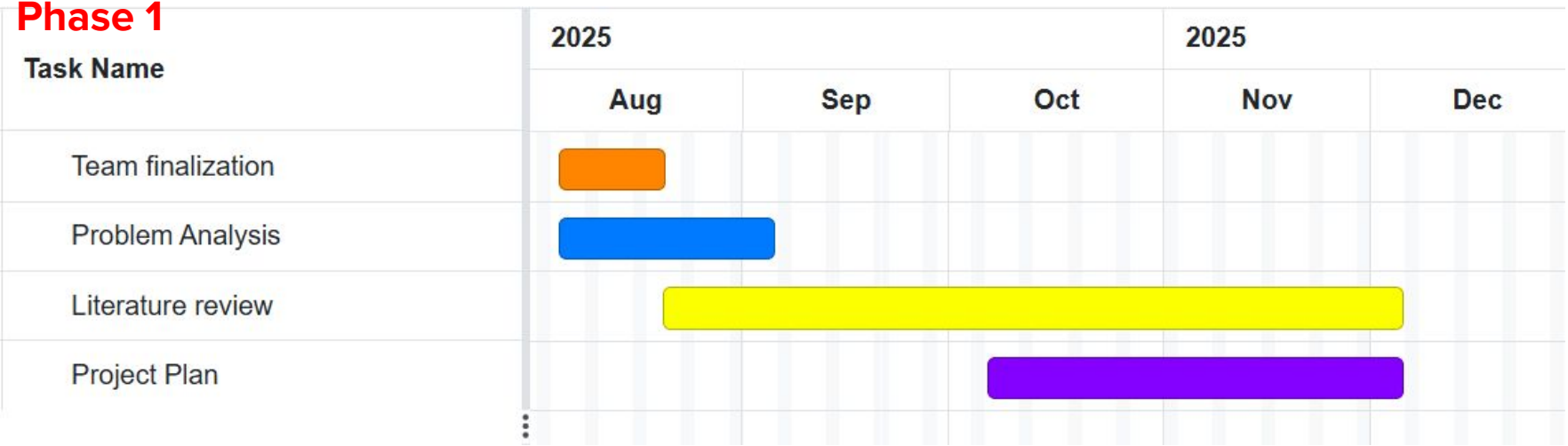
Title:	Effective, Platform-Independent GUI Testing (PIRLTEST, ACM 2024)
Methodology	Introduces PIRLTEST, a framework using Deep Reinforcement Learning (DQN) and **image embeddings** to test apps across all platforms (Android, Web, iOS). It operates **purely visually** and ignores all platform-specific code (like HTML or Android layouts). Creates a unique vector "fingerprint" of the app state by combining two analyses: 1. **Microscopic (widget embedding):** Uses computer vision (VGG-16) to find widgets and encode their visual appearance, location, and type. 2. **Macroscopic (layout embedding):** Organizes the overall structure into a layout tree, which is fed into an LSTM to create a layout vector. Uses a **curiosity-driven reward**, rewarding the agent for discovering novel states and interacting with new widgets to encourage high exploration coverage.
Findings/Research	The framework is truly platform-independent and works on mobile and web apps with zero adaptation cost. Achieved **significantly higher code coverage** and found more unique bugs than baseline testing tools.
Future work	(Inferred from methodology) Expand the visual "fingerprinting" to better understand complex custom UI elements, such as canvas or game engine UIs, where widget detection is difficult.
Datasets used	(Not specified in summary) Tested on a set of real-world mobile and web applications.
Gaps	**"Blind" to UX/Accessibility:** Like other RL papers, its goal is purely functional (code coverage, bug discovery). **No Heuristic Analysis:** It "sees" the layout and widgets but does not *evaluate* them for human-centric usability (e.g., "Is this a *good* layout?"). **Ignores DOM Structure:** The "purely visual" approach is powerful but misses structural information from the DOM (like ARIA tags) that is **critical** for accessibility testing. This directly justifies our project's **multimodal (Vision + Structure)** approach.

Title :	Using Vision LLMs for UI Testing (UW, 2025)
Methodology	Proposes an agent that uses a Vision (LLM) to test an application by inspecting screenshots, making tests resilient to underlying code changes. Operates on an agent loop:**1** Receives a natural language task (e.g., "click the settings icon"),**2** Takes a screenshot,**3** Sends the screenshot to the Vision LLM,**4** LLM decides an action (e.g., click coordinates),**5** Agent executes the action.The LLM also acts as the **test oracle**, checking the new screenshot to see if the task succeeded or failed.Tested three methods to give the LLM spatial awareness:**1** Grid overlay (imprecise),**2** Bounding boxes, and **3 Bounding boxes with OCR (winner)**, as Vision LLMs are highly optimized for text.
Findings/Research	Using **bounding boxes with OCR** (via OmniParser) gave the best UI element detection. However, the **LLM was an unreliable test oracle**, often hallucinating results — causing **false passes** and **false fails**.
Future work	(Inferred from findings) Improve the reliability of the LLM as a test oracle to reduce "hallucinated" states.Expand beyond simple click tasks to more complex, multi-step user flows.
Datasets used	A custom test setup using the Playwright browser automation tool.
Gaps	**Purely Functional Focus:** The LLM is only used to *execute* a task ("click the icon") and verify the *functional* outcome.**No UX/Accessibility Analysis:** It does **not** use its "vision" to analyze human-centric issues like color contrast, unreadable font sizes, or confusing layouts. This is a perfect gap for our project.**Reliability Issues:** The finding of "hallucinated" states shows a weakness in using LLMs as the sole test oracle.

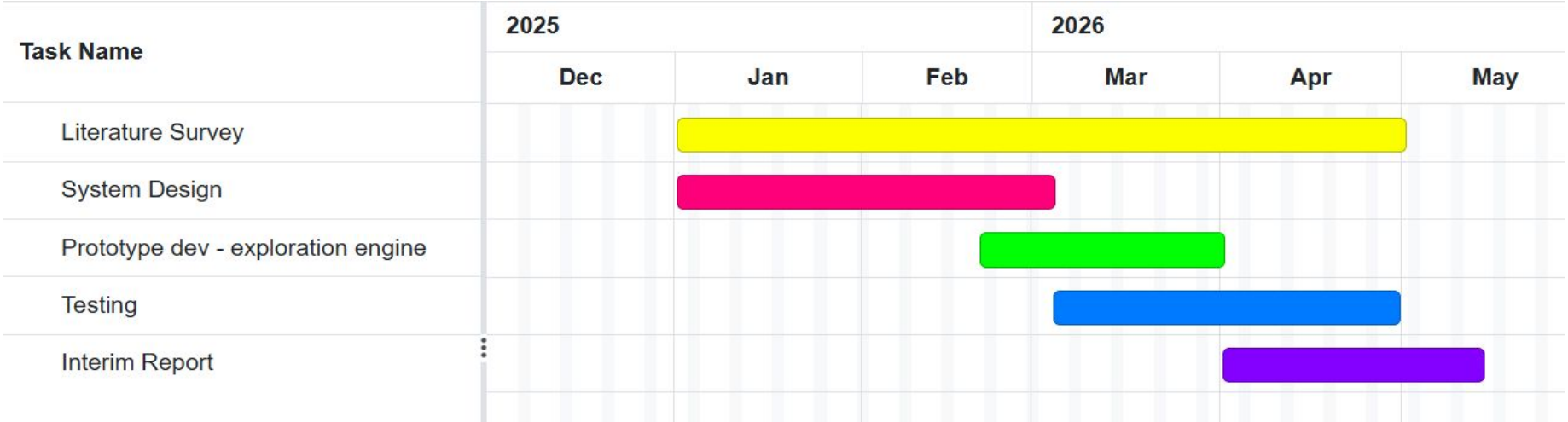
Title :	Self-Healing in Web Automation: Systematic Insights
Methodology	Self-healing in web automation is when test scripts use AI to automatically fix themselves when an application's UI changes. When a test fails to find an element, the system intelligently locates the correct one and updates the script on its own. This makes automated tests more resilient and significantly cuts down on manual maintenance.
Findings/Research	-Significant reduction in maintenance effort and false positives; improved test resilience -Real-time fault prediction and recovery demonstrated in production contexts
Core Techniques	- **Deep Learning Models**: CNNs for UI element recognition; RNNs for sequence prediction - **AI/Rule Hybrids**: Fuzzy matching, visual recognition, repository of element variants
Applications	Web and mobile app testing, cross-browser & dynamic UI environments; frameworks adapt to locator changes in real-world use.
Future work	- Improve ML transparency and explainability in locator adaptation - Combine AI with structured development practices (e.g., data-automation IDs) for robustness
Gaps	- ML applicability often limited or unclear; risk of “black-box” fixes - Self-healing may mask deeper QA weaknesses if not combined with solid test design

Gantt chart

Phase 1



Phase 2



Phase 3

Phase 4

Capstone (Phase-I , Phase-II and Phase III) Project Timeline

Expected Deliverables

- **Capstone I: Literature survey and architecture**
- **Capstone II: System Development & Test Generation**
  - Complete exploration loop: Playwright opens URL → captures DOM → LLM suggests next action → memory log built.
  - LLM-based test-case generator that converts exploration logs to structured executable Playwright test scripts.
  - Initial evaluation on multiple websites with metrics like number of tests generated, success rate, and coverage.

Capstone (Phase-I , Phase-II and Phase III) Project Timeline

Capstone III: Evaluation, Optimization & Paper Publication

Improved prompts and rules for test generation and assertion validation.

Quantitative evaluation: precision/recall of generated test cases vs manually written baseline.

Final dashboard visualization of results and full research paper (methodology, experiments, results, discussion, future work).

Capstone IV: Research Paper & Publication

- A comprehensive **research paper** is prepared, detailing the system's architecture, methodology, and the benchmark results from Capstone III.
- The final paper is submitted to a **Scopus-indexed conference or journal**.

Any other information

Provide any other information you wish to add on.

Note: Changes can be made in the template, with the consent of the guide for inclusion of any other information.

Conclusion and Future Work

Conclusion:

The project achieves a strong foundation in building an **AI-driven testing framework** that connects Playwright automation with LLM reasoning. It automates exploration, generates functional test cases, and performs UX analysis effectively. The system demonstrates that a **modular, agentic hybrid approach** can streamline test generation and improve coverage while minimizing manual effort.

Future Work :

- Conduct an in-depth **literature survey** to refine the exploration engine using recent multimodal LLM research.
- Enhance **system design** by adding scalability and modular improvements for better orchestration between the controller, LLM service, and dashboard.
- Begin **prototype development of the exploration engine**, focusing on adaptive navigation and improved context handling.
- Carry out **system testing** to validate functionality, precision, and robustness across varied web applications.
- **interim report** summarizing implementation progress, evaluation metrics, and planned optimizations for the next phase.

References

[1] C. Păduraru, R. Cristea, and A. Stefanescu, "End-to-end RPA-Like Testing Using Reinforcement Learning," in *Proc. 2024 IEEE International Conference on Software Testing, Verification and Validation (ICST)*, Toronto, ON, Canada, 2024, pp. 419–429.

[2] K. Turbeville, R. Kumar, J. Muengtaweepongsa, S. Stevens, and L. Li, "LLM-powered Multimodal Insight Summarization for UX Testing," in *Proc. ACM International Conference on Multimodal Interaction (ICMI)*, San Jose, Costa Rica, 2024. (Specific page numbers pending final publication.)

[3] R. F. Oguz, İ. Erdem, E. Olmezogullari, and M. S. Aktaş, "End-to-End Automated UI Testing Workflow for Web Sites with Intensive User-System Interactions," *Int. J. Softw. Eng. Knowl. Eng.*, vol. 32, no. 10, pp. 1297–1320, Aug. 2022.

[4] A. H. Mughal, "An Autonomous RL Agent Methodology for Dynamic Web UI Testing in a BDD Framework," *arXiv preprint arXiv:2503.08464*, Mar. 2025. (Reference to the arXiv pre-print is used until final publication details are confirmed.)

[5] A. Chevrot, A. Vernotte, J.-R. Falleri, and A. Cretin, "Are Autonomous Web Agents Good Testers?," *Proc. ACM Softw. Eng.*, vol. 2, no. ISSTA, pp. 206–228, Jun. 2025. (References a confirmed publication in the Proceedings of the ACM on Software Engineering.)

[6] J. Yoon, B. E. Maenpaa, K. Lee, and S. Yoo, "Intent-Driven Mobile GUI Testing with Autonomous Large Language Model Agents (DroidAgent)," in *Proc. 2024 IEEE International Conference on Software Testing, Verification and Validation (ICST)*, Toronto, ON, Canada, 2024, pp. 129–139.

Thank You